

Relatório do Sistema

Conciliador OFX × Planilha × Domínio — funcionalidades implementadas

1. Visão geral

Aplicativo desktop em Python/Tkinter para conciliação tripla:

- **Planilha (.xlsx)** com contas a pagar (vencimento, valor da parcela, NF, fornecedor, CNPJ, emissão).
- **Extrato bancário (OFX)** com pagamentos efetivados — só valores negativos (saídas) são considerados.
- **Sistema Domínio (Escrita Fiscal)** via ODBC (read-only) — puxa parcelas de NFe da tabela *bethadba.efentradaspar* com JOIN em *efentradas* e *effornece*.

O fluxo é: carregar as 3 fontes → casar Planilha × OFX (match exato + sugestões com tolerância) → cruzar resultado com o Domínio.

2. Leitura de dados

Planilha Excel (.xlsx) — `parser_xlsx.py`

- **Deteção automática da linha do cabeçalho** (procura nas primeiras 15 linhas, pula linhas em branco, títulos, logos).
- **Auto-deteção dos 6 campos obrigatórios por nome**, com dezenas de aliases: *Data vencimento* (vencimento, dt venc, prazo...), *Data emissão* (emissao, data nf, dt emis...), *Valor* (valor parcela, R\$, montante...), *Nº NF* (nf, nº nota, nº nf...), *CNPJ* (cnpj, cgc...), *Fornecedor* (fornecedor, razão social, beneficiário, favorecido...).
- **Fallback por conteúdo**: quando o nome não bate, classifica a coluna pelo tipo das células — mais datas → Data, mais números → Valor, textos mais longos → Fornecedor.
- **Diálogo de mapeamento manual** com preview ao vivo das primeiras 10 linhas; linhas em vermelho destacam quando alguma data ou valor não pôde ser convertido.
- **Conversão robusta de valores** no formato brasileiro: R\$ 1.234,56, vírgula decimal, parênteses para negativo.
- **Múltiplos formatos de data**: dd/mm/aaaa, aaaa-mm-dd, dd-mm-aaaa, dd/mm/aa, dd.mm.aaaa e células nativas de data do Excel.

Extrato OFX — `parser_ofx.py`

- Leitura via *ofxparse*.
- **Filtra automaticamente apenas pagamentos** (valores negativos), ignorando recebimentos — o app mostra a contagem de ignorados.
- Inverte o sinal para casar com a planilha (que tem valores positivos).

Sistema Domínio (ODBC) — `parser_dominio.py`

- **Padrão Janco**: credenciais em *data/dominio_config.json* (gitignored), separadas do *config.json* de preferências.
- **connect_dominio(readonly=True)** como context manager para scripts; **open_connection()** para conexões longas mantidas pela GUI.
- **load_odbc_config / save_odbc_config** para persistir e ler credenciais.
- **listar_empresas(conn)** lê *bethadba.geempre* com fallback entre variantes de nomes de coluna (RAZA_EMP / RAZAO_EMP / NOME_EMP).

- **listar_tabelas** filtra schema *bethadba* por padrão; checkbox no diálogo libera ver todos os schemas.
- **amostra** usa fallback TOP / FIRST / LIMIT para compatibilidade entre dialetos do SQL Anywhere.
- **extrair_pagamentos** aceita dois modos (tabela+WHERE ou SQL livre) e injeta automaticamente *CODI_EMP = ?* como parâmetro pyodbc.
- **Gotchas documentados:** DISTINCT + ORDER BY exige aliases do SELECT (não nomes qualificados — erro -854 no SA).

3. Modelo de dados

Campos obrigatórios em todas as fontes (planilha e Domínio):

- **Data vencimento** — chave de match com o OFX
- **Data emissão** — data do documento fiscal
- **Valor da parcela**
- **Número da NF**
- **CNPJ do fornecedor**
- **Nome do fornecedor**

O OFX é diferente: tem apenas *data de pagamento*, *valor* e *memo* (do banco). O memo aparece nas abas onde o OFX está envolvido.

Query padrão usada no Domínio

JOIN das 3 tabelas, parametrizado por empresa:

```
SELECT par.vcto_entp AS data_vencimento, par.vlor_entp AS valor_parcela, par.parc_entp AS
numero_parcela, ent.ddoc_ent AS data_emissao, ent.ume_ent AS numero_nf, forn.cgce_for AS
cnpj_fornecedor, forn.nome_for AS nome_fornecedor FROM bethadba.efentradaspar par JOIN
bethadba.efentradas ent ON ent.codi_emp = par.codi_emp AND ent.codi_ent = par.codi_ent
JOIN bethadba.efeornece forn ON forn.codi_emp = ent.codi_emp AND forn.codi_for =
ent.codi_for WHERE par.codi_emp = ?
```

4. Lógica de conciliação — matcher.py

- **conciliar_automatico**: match exato por (data, valor) quantizado para 2 casas. Casa duplicatas uma a uma.
- **gerar_sugestoes**: identifica pares de pendentes onde a diferença de data é ≤ 2 dias e a de valor é $\leq R\$ 10,00$. Ordena por proximidade (menor Δ primeiro).
- **conciliar_completo**: pipeline completo (match exato + sugestões).
- **diferenca(p, o)**: calcula Δ dias e Δ valor entre duas transações para exibição.

5. Interface gráfica (Tkinter) — main.py

Barra de ações (3 linhas no topo)

- **Linha 1 — Planilha**: Abrir planilha (.xlsx) | Editar colunas | label
- **Linha 2 — OFX**: Importar OFX | label
- **Linha 3 — Domínio**: Conectar Domínio | Selecionar empresa | Configurar fonte | Carregar pagamentos | label de status
- **Linha 4 — Ações**: Conciliar | Comparar com Domínio | resumo

Diálogo de mapeamento da planilha — DialogoMapeamento

- 6 dropdowns (todos obrigatórios) pré-selecionados com a sugestão automática.
- **Preview ao vivo** mostra as primeiras 10 linhas com as 6 colunas mapeadas — atualiza a cada mudança de dropdown.
- **Linhas em vermelho** quando vencimento, valor ou emissão não puderam ser convertidos.
- Status com contagem N/M linhas válidas.

Diálogos do Domínio — dialogos_dominio.py

- **DialogoConexao**: DSN + usuário + senha; testa a conexão (read-only) e persiste em *data/dominio_config.json*.
- **DialogoSelecionarEmpresa**: lista buscável de *bethadba.geempre* (código + razão social + CNPJ); duplo-clique seleciona.
- **DialogoFonte**: alterna entre *Tabela + WHERE* e *Query SQL manual*; lista tabelas do schema *bethadba* (com opção de ver outros schemas); preview até 100 linhas; mapeia os 6 campos.
- No modo SQL, quando há ? na query, o preview e a extração passam automaticamente o CODI_EMP da empresa selecionada como parâmetro pyodbc.

Abas do Notebook (7 abas em 2 blocos)

Bloco "Origem" — dados crus de cada fonte:

Bloco "Resultado" — fruto da conciliação:

#	Aba	Conteúdo	Ações
1	Planilha	Dados crus da planilha importada: linha, vencimento, emissão, valor, NF, CNPJ, fornecedor.	Inspeção visual
2	OFX	Pagamentos do extrato bancário: data de compensação, valor, memo do banco.	Inspeção visual

3	Domínio dados	Parcelas carregadas do banco do Domínio: vencimento, emissão, valor, NF, CNPJ, fornecedor.	Inspeção visual
4	Conciliados	Pares casados Planilha × OFX (verde = automático, azul = manual). Mostra Δ dias / Δ valor quando há diferença.	Desfazer conciliação manual
5	Pendentes	Dois lados: "Só na planilha" (com NF e Fornecedor) e "Só no OFX" (com Memo).	Conciliar selecionadas →
6	Sugestões	Pares aproximados Planilha × OFX ($\Delta \leq 2d$ e $\Delta \leq R\$ 10$), ordenados pelos mais próximos.	Aceitar como conciliação
7	Comparação	Cruzamento tripla Planilha × OFX × Domínio: verde (em tudo), amarelo (falta na contabilidade), vermelho (só no Domínio).	—

Comportamentos transversais

- Resumo no topo com contagem por categoria; títulos das abas mostram número entre parênteses (ex.: *Planilha (142)*).
- Carregar nova planilha/OFX **limpa automaticamente** os resultados anteriores de conciliação.
- Aviso quando o mapeamento produz **0 lançamentos**.
- Aviso quando **Carregar pagamentos** sem empresa selecionada — evita puxar dados de todas as empresas misturadas.
- Conciliação **manual** entre pendentes (selecionar 1 de cada lado e clicar "Conciliar selecionadas →").
- **Aceitar sugestão** promove um par aproximado para conciliado.
- **Desfazer conciliação manual** devolve o par para pendentes e recalcula sugestões.

6. Persistência e configuração

- **config.json** (raiz do projeto, gitignored): preferências do app — mapeamento de colunas da planilha, empresa selecionada do Domínio, fonte do Domínio (modo, tabela ou SQL, WHERE, mapeamento das 6 colunas).
- **data/dominio_config.json** (gitignored): credenciais ODBC — DSN, usuário, senha em texto.
- **Migração automática:** configs antigas com `cfg["dominio"]` misturado são divididas automaticamente na primeira execução do app atualizado.

7. Infraestrutura

- **requirements.txt:** openpyxl, ofxparse, pyodbc, reportlab.
- **.gitignore:** protege `config.json`, `data/dominio_config.json`, `.env/`, `__pycache__`/, planilhas, OFX e CSV.
- **README.md:** guia completo de instalação, configuração do DSN ODBC para o Domínio (SQL Anywhere 17), fluxo de uso, convenções do projeto Janco.
- **gerar_pdf.py:** script para regenerar este relatório.
- **Repositório:** github.com/Fernandovini2607/conciliador (privado).

8. Convenções herdadas do projeto Janco

- Toda tabela do Domínio Escrita Fiscal vive no schema **bethadba** (sempre prefixar).
- Conexão ODBC é sempre **read-only** (o usuário ODBC do Domínio não tem GRANT de UPDATE; qualquer escrita exige RPA — fora do escopo deste app).
- **Gotcha SQL Anywhere:** DISTINCT + ORDER BY exige usar os aliases do SELECT, não nomes qualificados (erro -854 com `p.CODI_GRU`; use `grupo` se aliasou).
- Empresas vivem na mesma base; separação é por **CODI_EMP** em cada tabela. O app injeta esse filtro automaticamente.

9. O que ainda não existe

- Tolerância configurável (hoje fixa em 2 dias / R\$ 10,00).
- Exportação do resultado para Excel/PDF/CSV (com cores).
- Tratamento de planilhas com colunas separadas de Crédito/Débito.
- Match por similaridade textual (descrição ou fornecedor).
- Suporte a múltiplas contas no mesmo OFX.
- Histórico de conciliações entre execuções.
- Conciliação manual diretamente na aba do Domínio.
- Escrita de volta no Domínio (exigiria RPA via pywinauto).